

Gymnázium, Praha 6, Arabská 14

vyučující Mgr. Jan Lána

Devil's pact

Jonáš Frk, Andrii Hryb, Mark Zazdravnykh 3.E



6. března 2026

Poděkování

Rádi bychom poděkovali našim rodinám a blízkým za jejich podporu a motivaci, které bylo při celém procesu potřeba více než dostatek.

Anotace

Tato práce se zabývá implementací 2D hry roguelike stylu. Hlavním cílem našeho projektu bylo vytvořit hru v programovacím jazyce Java s využitím grafického frameworku JavaFX. Hra obsahuje předem generovaná prostředí a jedinečný systém bojů založený na hodech kostkami, který funguje na prvku náhody s kombinací ovlivnění hodů kostek. Součástí práce je popis hry a herní logiky, vytváření vlastních postav a textur pro hru a řešení komplikací vývoje. Výsledkem je hra obsahující deset úrovní se systémem obchodu a pasivními vylepšeními.

Abstract

This thesis deals with the implementation of a 2D roguelike style game. The main goal of our project was to create a game in the Java programming language using the JavaFX graphics framework. The game contains pre-generated environments and a unique dice-based combat system that works on an element of chance with a combination of dice roll effects. The thesis includes a description of the game and game logic, creating custom characters and textures for the game, and solving development complications. The result is a game containing ten levels with a store system and passive upgrades.

Prohlášení

Prohlašujeme, že jsme ročníkovou práci vypracovali samostatně pod vedením Mgr. Jana Lány a že jsme uvedli všechny použité informační zdroje a literaturu v souladu se zásadami citování.

V Praze dne:

Vlastnoruční podpisy autorů:

Jonáš Frk:

Andrii Hryb:

Mark Zazdravnykh:

Obsah

1. Úvod.....	6
1.1. Cíl práce.....	6
1.2. Motivace a přínos naší ročníkové práce.....	6
2. Problémy při vývoji a jejich řešení.....	7
2.1. Chyby načítání textur (NullPointerException).....	7
2.2. Synchronizace animací a výpočtů.....	7
2.3 Správa více nepřátel (Duo fights).....	7
3. Grafické zpracování a vizuální styl.....	8
3.1. Tvorba pixel artu v programu Aseprite.....	8
3.2. Technické aspekty a vykreslování.....	8
4. Technické mechanismy a architektura.....	9
4.1. Předběžné načítání (Pre-caching).....	9
4.2. Stavový automat (state machine).....	9
4.3. Procedurální progresse.....	9
5. Herní mechaniky a ekonomika.....	10
5.1. “Gamble” soubojová mechanika.....	10
5.2. Ekonomika a vylepšení.....	10
6. Ukázky kódu.....	11
6.1. Sekvencování asynchronních animací.....	11
7. Závěr a budoucí rozvoj.....	13

1. Úvod

1.1. Cíl práce

Cílem této práce bylo vytvořit funkční 2D roguelike hru, kde se hráč bude procházet generovaným prostředím. Hlavním úkolem je úspěšně projít všemi deseti patry dungeonu a porazit hlavního bosse- Herní mechanika stojí na průzkumu mapy a soubojům s protivníky založeném na náhodě (hodem kostkami), který lze ovlivnit pasivními vylepšeními.

1.2. Motivace a přínos naší ročníkové práce

Hlavní motivací byla snaha zkombinovat napětí casino mechanik s RPG prvky. Projekt také slouží v prohloubení znalostí v oblastech: Objektově orientovaného programování v jazyce Java. Práce s grafickým frameworkem JavaFX Tvorba vlastních obrázků, bossů a textur a následně jejich implementace do hry.

2. Problémy při vývoji a jejich řešení

Během vývoje jsme narazili na několik logických a architektonických překážek, které vyžadovaly specifické technické řešení:

2.1. Chyby načítání textur (NullPointerException)

Původní kód balil načítání obrázků do prázdných bloků. Při překlepu v cestě hra nespada, ale vykreslovala jen prázdné čtverce. Řešením bylo sjednocení struktury složek, důsledná kontrola velikosti písmen (case-sensitivity) a implementace logování chyb do konzole.

2.2. Synchronizace animací a výpočtů

V soubojovém systému se zranění často odečetlo dříve, než postava vizuálně zaútočila. Protože JavaFX běží na UI vlákne, nebylo možné použít `Thread.sleep()`. Aplikovali jsme asynchronní časování pomocí objektů `Timeline`, kde se životy odečtou až v konkrétním `KeyFrame` ve chvíli vizuálního kontaktu.

2.3 Správa více nepřátel (Duo fights)

Při přidání dvou nepřátel do místnosti se jejich animace (idle sway) začaly rušit. Logiku jsme museli rozdělit do nezávislých `Timeline` objektů a vytvořit pole pro uchovávání životů a pozic každé entity zvlášť.

3. Grafické zpracování a vizuální styl

3.1. Tvorba pixel artu v programu Aseprite

Veškerá grafika byla vytvořena v editoru Aseprite. Zvolili jsme styl s rozlišením 100x100 pixelů pro běžné objekty, což evokuje retro dungeon hry. Pro klíčové bossy jsme využili rozlišení 200x200 až 300x300 pixelů, aby vynikly detaily.

Postavy: Disponují animacemi pro útok a pohyb, složenými z několika po sobě jdoucích snímků.

Prostředí: Sada dlaždic (tileset) obsahuje zdi, podlahy, truhly a schodiště.

Efekty: Klíčovým prvkem jsou animace házení kostek, které vizualizují mechaniku souboje.

3.2. Technické aspekty a vykreslování

Využili jsme vrstevný systém (Group a ImageView). To nám umožnilo efektivně oddělit statické pozadí, herní entity a UI, čímž jsme zamezili vizuálním chybám (prolínání objektů) při pohybu postavy.

4. Technické mechanismy a architektura

4.1. Předběžné načítání (Pre-caching)

Všechny textury se načítají do vícerozměrných polí hned při startu. Tím se šetří výkon a zabraňuje se „zasekávání“ hry při přechodu mezi místnostmi.

4.2. Stavový automat (state machine)

Hra plynule přepíná mezi stavy „Průzkum dungeonu“ (viditelná mapa) a „Uvnitř místnosti“ (bojové pozadí, obchod) pomocí proměnné inRoom.

4.3. Procedurální progresse

Hra obsahuje 10 pater (floorLevel). S každým patrem se zvyšuje náročnost – od 4. patra se objevují „Duo“ souboje a na 10. patře čeká finální boss. Algoritmus generování mapy zajišťuje, že každé patro je unikátní.

5. Herní mechaniky a ekonomika

5.1. “Gamble” soubojová mechanika

Hlavním bodem je systém tří kostek:

Útočná kostka: Určuje sílu poškození nepřítele.

Léčící kostka: Rozhoduje o obnově životů hráče.

Štítová kostka: Generuje dočasné brnění pro další kolo, protivníkův útok se nejdříve bude odečítat ze štítu a až poté z životů.

5.2. Ekonomika a vylepšení

Za poražené nepřátele hráč získává mince, které utrací v náhodně se objevujícím obchodě. Předměty (např. Lifesteal – léčení útokem) jsou uloženy v poli hasItem a dynamicky ovlivňují matematiku souboje.

6. Ukázky kódu

6.1. Sekvencování asynchronních animací

Jedním z nejvýznamnějších a nejčastěji opakovaných principů ve hře je sekvencování asynchronních animací. Ukázka níže reprezentuje logiku vizuálního útoku nepřítele:

```
544 Timeline getAttackAnim(ImageView view, int eId) {
545     double startX = view.getTranslateX();
546     Timeline anim = new Timeline();
547     Image base;
548     Image a1;
549     Image a2;
550     Image a3;
551     Image a4;
552
553     if (eId >= 1 && eId <= 8) {
554         base = ENEMY_ATTACK_IMGS[eId - 1][0];
555         a1 = ENEMY_ATTACK_IMGS[eId - 1][1];
556         a2 = ENEMY_ATTACK_IMGS[eId - 1][2];
557
558         if (eId == 1 || eId == 3 || eId == 6 || eId == 8) {
559             anim.getKeyFrames().addAll(
560                 new KeyFrame(drtn: Duration.ZERO, e -> {
561                     if (a1 != null) view.setImage(value: a1);
562                 }),
563                 new KeyFrame(drtn: Duration.seconds(s: 0.1), e -> {
564                     view.setTranslateX(startX - 15);
565                 }),
566                 new KeyFrame(drtn: Duration.seconds(s: 0.2), e -> {
567                     if (a2 != null) view.setImage(value: a2);
568                 }),
569                 new KeyFrame(drtn: Duration.seconds(s: 0.3), e -> {
570                     view.setTranslateX(value: startX);
571                 }),
572                 new KeyFrame(drtn: Duration.seconds(s: 0.4), e -> {
573                     if (a1 != null) view.setImage(value: a1);
574                 }),
575                 new KeyFrame(drtn: Duration.seconds(s: 0.5), e -> {
576                     if (base != null) view.setImage(value: base);
577                 })
578             );
579         }
```

Proč je to důležité: Tento princip se ve hře využívá všude. Místo toho, aby se hardcodoval každý pixel pohybu, vytvořila se univerzální metoda, která přijme grafický objekt (ImageView), zjistí jeho počáteční pozici a poté k němu "přilepi" časovou osu. Po skončení animace se kontrola plynule vrátí zpět hráči.

7. Závěr a budoucí rozvoj

Projekt úspěšně splnil všechna naše stanovená zadání, ale také zanechal prostor pro rozšíření do budoucna. V budoucnu plánujeme rozšíření o další levely, aby hra byla na déle, jelikož v současné verzi se hra dokáže dokončit do přibližně deseti minut. Dále plánujeme přidat více protivníků s unikátními aby hra byla náročnější. Také plánujeme vytvořit více objektů se kterými může hráč interagovat, například nové vylepšení na boje atd.

Zdroje

- JavaFX a oficiální dokumentace (Základ enginu)
- JavaFX AnimationTimer (Herní smyčka):
 - Odkaz:
<https://openjfx.io/javadoc/11/javafx.graphics/javafx/animation/AnimationTimer.html>
- JavaFX Timeline a KeyFrame (Bojové animace):
 - Odkaz:
<https://openjfx.io/javadoc/11/javafx.graphics/javafx/animation/Timeline.html>
- Návrhové vzory pro hry (Game Architecture)
- Vzor "Game Loop" (Herní smyčka):
 - Odkaz:
<https://gameprogrammingpatterns.com/game-loop.html>
- Tvorba her v JavaFX (Tuts+ Tutorial):
 - Odkaz:
<https://code.tutsplus.com/introduction-to-javafx-for-game-development--cms-23835t>
- Algoritmy pro generování map (Procedural Generation)
- Metoda generateDungeon(), která náhodně sází místnosti a spojuje je chodbami, je klasickým postupem z žánru Roguelike her:
 - RogueBasin - Dungeon-Building Algorithm:
 - Odkaz:
http://www.roguebasin.com/index.php/Dungeon-Building_Algorithm
 - Procedurální generování (Spelunky Generator Lessons):
 - Odkaz:
<https://tinysubversions.com/spelunkyGen/>